

Natural-Language Safety Constraints for STL-Constrained Reinforcement Learning: A Problem Definition

Anonymous

Abstract

We define a focused problem at the interface of natural-language task specification, Signal Temporal Logic (STL), and safe reinforcement learning. The problem is not offline trajectory synthesis from a known model, nor open-domain language grounding in full generality. Given an instruction containing both a goal and a safety requirement, the objective is to translate the safety-relevant portion into a typed STL constraint and use it as a runtime safety specification for an RL agent. The object of study is whether the resulting STL constraint reduces trajectory-level safety violations during learning and deployment while preserving task performance.

1 Motivation

Reinforcement learning (RL) agents are increasingly expected to follow high-level user instructions in continuous environments. In safety-critical domains, these instructions often mix task objectives with constraints, for example: “reach the target while always avoiding the red zone and keeping speed below 1.0.” A reward-only interpretation is insufficient, since an agent may obtain high return while temporarily violating the safety requirement. STL provides a formal language for expressing such requirements over real-valued trajectories, making it possible to monitor or constrain behavior in a temporally precise way.

2 Setting

Consider an episodic Markov decision process with horizon T , continuous state space $\mathcal{S}$, action space $\mathcal{A}$, and policy $\pi(a \mid s)$. A trajectory is denoted by

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T).$$

The environment is equipped with a typed predicate schema $\mathcal{P}$ over states and signals, such as

$$\text{goal}(s), \quad \text{red}(s), \quad v(s) \leq v_{\max}, \quad d(s, \text{obs}) \geq d_{\min}.$$

The user provides a natural-language instruction L containing a task objective and at least one safety clause. We assume the predicate schema is known, but the mapping from the safety clause in L to an STL formula must be produced by the system.

3 Natural Language to STL Safety

Let T_{safe} denote a translator that extracts and formalizes the safety-relevant content of L :

$$\varphi_{\text{safe}} = T_{\text{safe}}(L, \mathcal{P}).$$

For the instruction “go to the target, but always avoid the red zone and keep speed below 1.0,” a possible decomposition is

$$\varphi_{\text{goal}} = F_{[0,T]} \text{goal},$$

$$\varphi_{\text{safe}} = G_{[0,T]} (\neg \text{red} \wedge v \leq 1.0).$$

The goal formula describes what the agent should accomplish; the safety formula describes conditions that should not be violated along the trajectory. We focus on the latter. The STL satisfaction relation $\tau \models \varphi_{\text{safe}}$ defines whether the complete trajectory is safe. When robust STL semantics are available, $\rho(\tau, \varphi_{\text{safe}})$ gives a signed safety margin, where positive values indicate satisfaction and negative values indicate violation.

4 Problem Definition

The problem is to learn or execute a policy π that optimizes task performance while respecting a safety formula induced from natural language:

$$\max_{\pi} \mathbb{E}_{\tau \sim \pi} [R_{\text{goal}}(\tau)] \quad \text{s.t.} \quad \Pr_{\tau \sim \pi} [\tau \models \varphi_{\text{safe}}] \geq 1 - \delta.$$

Equivalently, in a hard-safety deployment setting, the desired property is

$$\tau \models \varphi_{\text{safe}} \quad \text{for every executed trajectory } \tau.$$

The central research question is: given safety constraints generated from natural language rather than manually written by an expert, can an RL system use the resulting STL specification to reduce safety violations without making the task infeasible or destroying goal performance?

5 Scope

This definition intentionally restricts the first stage of study. We do not require open-vocabulary instruction following, autonomous repair of infeasible specifications, or full natural-language task planning. We assume a finite predicate schema and focus on safety clauses whose STL forms are monitorable during execution. The downstream mechanism may be a monitor, shield, constrained RL algorithm, or robustness-based penalty.

6 Key Challenges

- **Grounding.** Safety phrases such as “avoid,” “safe distance,” or “slowly” must be mapped to typed predicates, thresholds, and intervals.
- **Predictive safety.** Current-state monitoring is insufficient when an action can make future violation unavoidable.
- **Learning-time risk.** Exploration can violate safety before convergence, so training safety and deployment safety must be separated.
- **Uncertainty and conflict.** Runtime enforcement may use imperfect dynamics or perception, and the natural-language goal may be infeasible under hard safety.
- **Downstream evaluation.** Translation should be judged by trajectory-level violation rate, robustness margin, and task success, not exact match alone.